

Table 1.1 Currently existing applications of (variants of) CMSA

Optimization problem	Publication	Year
Minimum common string partition	[12, 16, 18]	2016, 2020, 2021
Minimum covering arborescence	[18]	2016
Repetition-free longest common subsequence	[13, 14]	2016, 2018
Multi-dimensional knapsack	[16, 17, 69]	2016, 2017, 2021
Maximising the net present value of project schedules	[98]	2019
Binary optimization	[15]	2019
Minimum capacitated dominating set	[83]	2019
Routing cooperative air-ground robots with fuel constraints	[8]	2019
Maximum happy vertices problem	[49, 66, 100]	2019, 2022
Score-constrained packing	[54]	2020
Refueling and maintenance planning of nuclear power plants	[39]	2021
Test data generation in software product lines	[44]	2021
Bus driver scheduling	[90]	2022
Minimum positive influence dominating set	[4]	2022
Unit disk cover	[5]	2022
Electric vehicle routing	[2, 3]	2022, 2023
Multi-way multi-dimensional number partitioning	[37]	2023
Closest String	[79]	2023
Maximum disjoint dominating set	[88, 89]	2023, 2024
Rooted max tree coverage	[104]	2024

1.3.1 *Standard CMSA*

We start our introduction to CMSA by describing a standard algorithm variant, which is henceforth simply called CMSA. However, before doing so, the general idea of CMSA is briefly described. At each iteration, CMSA first generates several (generally valid) solutions to the tackled problem instance in a probabilistic way. Next, the components of these solutions are added to an initially empty sub-instance C' . This sub-instance is then passed to an exact solver (if convenient, this can be an ILP solver) which delivers the best solution found in a limited time. Based on this solution, the incumbent sub-instance is adapted. In particular, based on an aging mechanism, seemingly useless solution components are removed from C' in order not to slow down the solver in the next algorithm iteration.

The first action that needs to be taken for applying CMSA to a combinatorial optimization problem is defining the set C of solution components, that is, those components of which solutions to the considered problem are composed. Later we will provide specific examples for such a definition. For the moment, however, let $C = \{c_1, \dots, c_n\}$ be a generic set of solution components. Moreover, any valid solution $S \in \mathcal{S}$ to the considered optimization problem—where $\mathcal{S}$ is the set of all valid solutions—can be expressed as a subset of C , that is, $S \subseteq C$ for all $S \in \mathcal{S}$.

Algorithm 1.1: Pseudo-code of standard CMSA

```

1: input 1: Complete set of solution components  $C$ 
2: input 2: values for CMSA parameters  $n_a$ ,  $age_{\max}$ , and  $t_{\text{ILP}}$ 
3:  $S^{\text{bsf}} := \emptyset$ 
4:  $C' := \emptyset$ 
5:  $age[c] := 0$  for all  $c \in C$ 
6: while CPU time limit not reached do
7:   for  $i := 1, \dots, n_a$  do
8:      $S := \text{ProbabilisticSolutionConstruction}(C)$ 
9:     if  $f(S) < f(S^{\text{bsf}})$  then  $S^{\text{bsf}} := S$  endif
10:    for all  $c \in S$  and  $c \notin C'$  do
11:       $age[c] := 0$ 
12:       $C' := C' \cup \{c\}$ 
13:    end for
14:  end for
15:   $S^{\text{ILP}} := \text{SolveSubinstance}(C', t_{\text{ILP}})$ 
16:  if  $f(S^{\text{ILP}}) < f(S^{\text{bsf}})$  then  $S^{\text{bsf}} := S^{\text{ILP}}$  end if
17:   $\text{Adapt}(C', S^{\text{ILP}}, age_{\max})$ 
18: end while
19: output:  $S^{\text{bsf}}$ 

```

Finally, let $f : \mathcal{S} \mapsto \mathbb{N}^+$ for the following discussion be the objective function to be minimized, and let $f(\emptyset) := \infty$.

Algorithm 1.1 provides the pseudo-code of the standard version of CMSA. The algorithm starts by initializing both the best-so-far solution S^{bsf} and the sub-instance C' , which is always a subset of C , to the empty set. Furthermore, the so-called age values of all solution components are initialized to zero, that is, $age[c] := 0$ for all $c \in C$. The main loop of CMSA consists of four actions.

1. First, in the **construct step** of CMSA, a number of n_a valid solutions to the considered problem are probabilistically generated (see line 8 of Algorithm 1.1).
2. Second, in the **merge step** of CMSA, the current sub-instance C' is updated with the solution components found in these n_a solutions (see lines 10–13). That is, those solution components that (1) are found in at least one of the n_a constructed solutions and (2) do currently not form part of C' , are added to C' and their age value is set to zero.
3. Third, in the **solve step** of CMSA, an ILP solver is applied in order to find the best possible solution that only contains components from sub-instance C' , within a time limit of t_{ILP} CPU seconds (see line 15).
4. Fourth, in the **adapt step** of CMSA, sub-instance C' is adapted based on the solution S^{ILP} returned by the ILP solver; see function $\text{Adapt}(C', S^{\text{ILP}}, age_{\max})$ in line 17. In particular, in this function, sub-instance C' is adapted in the following way. First, the age values of all components in $C' \setminus S^{\text{ILP}}$ are incremented by one. Second, the age values of all components in S^{ILP} are set to zero. The final action in the *adapt step* consists of removing all those components from C' whose age value has reached the maximum allowed age ($age_{\max}$). This is done in order to

prevent components that never appear in S^{ILP} from slowing down the ILP solver in subsequent CMSA iterations. Note that the age value $\text{age}[c]$ of a solution component $c \in C$, at any time, indicates the number of consecutive CMSA iterations for which c has formed part of sub-instance C' without having been included in the ILP-solution to the sub-instance C' .

These four steps are iterated until a given CPU time limit is reached. The output of the algorithm is S^{bsf} , the best solution found during the whole process.

Note that the *construct step* and the *solve step* of such a CMSA algorithm are problem-dependent. In particular, for the *construct step* of the algorithm, generally, a greedy heuristic for the tackled problem is used in a randomized way, and the *solve step* depends on the availability of an ILP model for the tackled problem. Moreover, the way in which an ILP model is exactly generated based on sub-instance C' leaves room for variation. In contrast, the *merge step* and the *adapt step* are problem-independent.

1.4 Application to Minimum Dominating Set

In the following, we show how standard CMSA can be applied to the so-called Minimum Dominating Set problem. In particular, we will show two different ways of defining the set of solutions components, which results in slightly different ways of generating the ILP models corresponding to the sub-instances at each algorithm iteration.

Requirements for the Application of CMSA

In order to apply the standard CMSA algorithm from the previous section (see Sect. 1.3.1) to any combinatorial optimization problem, we need to define two algorithm components:

1. A probabilistic way of generating valid solutions
2. An ILP model of the tackled problem

The Minimum Dominating Set (MDS) problem is a well-known combinatorial optimization problem in Computer Science. Given an undirected graph $G = (V, E)$, where $V = \{v_1, \dots, v_n\}$ is the set of n vertices and E is the set of edges, the goal of the problem is to find a smallest dominating set $D^* \subseteq V$. Hereby, a set $D \subseteq V$ is called a *dominating set* of G if and only if for every vertex $v \in V$ it holds that

1. $v \in D$, or
2. $\exists v' \in D$ such that $(v, v') \in E$.

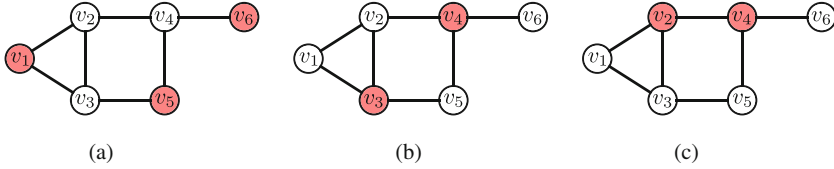


Fig. 1.7 An undirected graph with three different MDS solutions. **(b)** and **(c)** show alternative optimal solutions. **(a)** Sub-optimal solution. **(b)** An optimal solution. **(c)** Another optimal solution

In other words, a subset of the vertices of graph G is called a dominating set of G if every vertex of G is either in the set or adjacent to at least one vertex from the set; see Fig. 1.7 for an example. In this context, note that the set of neighbors of v in G is denoted by $N(v)$, that is, $N(v) := \{v' \in V \mid (v, v') \in E\}$. Moreover, $N[v] := N(v) \cup \{v\}$ is called the *closed neighborhood* of v .

The standard ILP model for the MDS problem makes use of a binary variable x_i for each vertex $v_i \in V$. The model can be stated as follows.

$$\min \sum_{v_i \in V} x_i \quad (1.13)$$

$$\begin{aligned} \text{subject to} \quad & \sum_{v_j \in N(v_i)} x_j + x_i \geq 1 & \forall v_i \in V \\ & x_i \in \{0, 1\} & \forall v_i \in V \end{aligned} \quad (1.14)$$

The constraints (1.14) make sure that any valid solution either contains a node $v_i \in V$ or at least one of its neighbors.

Finally, note that the MDS problem is known to be NP-hard, meaning that there is no known polynomial-time algorithm to solve it for arbitrary graphs unless $P = NP$. As a result, research often focuses on finding efficient algorithms for special cases or developing heuristic algorithms.

1.4.1 An Intuitive Way of Defining the Solution Components

> Programming Code Availability of This Application

Note that the C++ program code for this way of applying standard CMSA to the MDS problem is included in Appendix A of this book.

The first important step for the development of any CMSA algorithm is the definition of the set of solution components (C). In the case of the MDS problem,

Algorithm 1.2: MDS solution construction

```

1: input: a graph  $G = (V, E)$ 
2:  $s := \emptyset$ 
3:  $\bar{V} := \{v_i \in V \mid N[v_i \mid s] \neq \emptyset\}$ 
4: while  $\bar{V} \neq \emptyset$  do
5:    $v_j := \text{ChooseFrom}(\bar{V})$ 
6:    $s := s \cup \{v_j\}$ 
7:    $\bar{V} := \{v_i \in V \mid N[v_i \mid s] \neq \emptyset\}$ 
8: end while
9: output: a valid solution  $s$ 

```

there is an intuitive way of doing so. Hereby, set C simply contains a component c_i for each vertex $v_i \in V$. Therefore, for the following discussion, remember that v_i and c_i both refer to vertex v_i of the input graph G . The standard CMSA for the MDS problem based on the intuitive set of solution components is henceforth labeled CMSA_INT.

1.4.1.1 Constructing Solutions to the MDS Problem

In the following, we say that, if a vertex v_i is added to a solution s under construction, then v_i *covers itself and all its neighbors*, that is, it covers v_i (itself) and all $v_j \in N(v_i)$. Moreover, assuming that s is a partial solution under construction, we denote by $N[v_i \mid s] \subseteq N[v_i]$ the *set of uncovered vertices* (with respect to s) from the closed neighborhood $N[v_i]$ of $v_i \in V$.

The solution construction mechanism is shown in Algorithm 1.2. It starts with an empty solution $s = \emptyset$. For each construction step, $\bar{V} \subseteq V$ is defined as the set of vertices that can cover at least one of the vertices not covered yet by s . At each construction step, exactly one vertex is chosen from $\bar{V}$ in function $\text{ChooseFrom}(\bar{V})$. This works as follows. First, a random number r is sampled uniformly at random from $[0, 1]$. In case $r \leq d_{\text{rate}}$, $v_j \in \bar{V}$ is chosen such that

$$v_j := \max_{v_i \in \bar{V}} \{|N[v_i \mid s]|\} \quad (1.15)$$

Otherwise—that is, in case $r > d_{\text{rate}}$ —a number of $\min\{l_{\text{size}}, |\bar{V}|\}$ vertices from $\bar{V}$ is pre-selected and placed into a candidate set $L \subseteq \bar{V}$ such that

$$|N[v_i \mid s]| \geq |N[v_k \mid s]| \quad \text{for all } v_i \in L, v_k \in \bar{V} \setminus L \quad (1.16)$$

A vertex $v_j \in L$ is then chosen uniformly at random. Finally, note that each solution s , after finalizing its construction, is converted into a corresponding solution S that contains for each $v_i \in s$ the corresponding solution component c_i .

1.4.1.2 Solving Sub-instances of the MDS Problem

Next, we will describe how to use the ILP model of the MDS problem to solve a sub-instance C' within CMSA_INT. In this context, remember that—in any CMSA algorithm—a sub-instance C' contains those solution components that may appear in valid solutions to C' .

In order to solve a sub-instance C' by means of the application of an ILP solver in function `SolveSubinstance( $C'$ ,  $t_{\text{ILP}}$ )` of Algorithm 1.1, the MDS ILP model is extended in CMSA_INT by adding the following set of constraints:

$$x_i = 0 \quad \text{for all } c_i \in C \setminus C' \quad (1.17)$$

In this way, only solution components (resp. vertices) that form part of the sub-instance C' may be selected to appear in solutions.

Note that the function `SolveSubinstance( $C'$ ,  $t_{\text{ILP}}$ )`—see line 15 of Algorithm 1.1—returns the ILP solver solution in terms of a set S^{ILP} of solution components. Moreover, the application of the ILP solver is subject to a time limit of t_{ILP} CPU seconds, which means that the solution S^{ILP} returned by the function `SolveSubinstance( $C'$ ,  $t_{\text{ILP}}$ )` is not necessarily an optimal solution to the sub-instance C' .

➤ ILP Solver Dependent Settings Considered in This Book

Please be aware that while a commercial ILP solver like CPLEX (utilized throughout this book) can be used with its default parameter settings, this may not always yield the best results. Therefore, we explore the following options to possibly enhance performance beyond the default settings:

1. **Heuristic emphasis:** CPLEX offers a parameter to balance between the speed of proving optimality and the speed to improve the best-found solution during execution. As proving optimality is not a priority in CMSA algorithms, we consider a parameter $\text{cplex}_{\text{emphasis}} \in \{\text{true}, \text{false}\}$, where $\text{cplex}_{\text{emphasis}} = \text{false}$ refers to the default setting of CPLEX and $\text{cplex}_{\text{emphasis}} = \text{true}$ indicates a setting of the emphasis parameter to a value of five (highest heuristic emphasis value).
2. **Warm start:** When started in default mode, an ILP solver initially does not know any valid solution to the tackled problem. In some cases, providing the ILP solver with an initial valid solution can speed up the solving process. In the context of CPLEX, this is called a warm start. We consider a parameter $\text{cplex}_{\text{warmstart}} \in \{\text{true}, \text{false}\}$, where $\text{cplex}_{\text{warmstart}} = \text{false}$ does not provide an initial solution to CPLEX, whereas $\text{cplex}_{\text{warmstart}} = \text{true}$ provides the best-so-far solution S^{bsf} to CPLEX as initial solution.
3. **Aborting a CPLEX call:** Even with an increased value for the heuristic emphasis, CPLEX spends computation time on bound computations that eventually lead to proving optimality. In some applications, a lot of computation time might be spent on these efforts. In fact, in our experience, it is sometimes beneficial to

abort CPLEX when the first solution is found that improves over the best-so-far solution S^{bsf} . This CPLEX behavior is invoked in the applications of this book with a parameter $\text{cplex}_{\text{abort}} \in \{\text{true}, \text{false}\}$.

This completes the description of the application of standard CMSA based on the intuitive set of solution components to the MDS problem.

1.4.2 A Generic Way of Defining the Solution Components

Depending on the optimization problem to be solved, sometimes, there might be no intuitive way of defining the set of solution components. However, a generic way of doing so works as follows. For this purpose, we assume to have a binary ILP model for the tackled problem at hand.¹⁴ The set C of generic solution components will contain for each binary variable x_i two solution components:

1. Component c_i^0 : corresponding to $x_i = 0$.
2. Component c_i^1 : corresponding to $x_i = 1$.

In case a solution s to the problem is characterized by $x_i = 0$, the corresponding CMSA-solution S contains component c_i^0 . Similarly, if $x_i = 1$ in a solution s , then S contains component c_i^1 . In the case of the MDS problem, for example, the generic set of solution components C is defined as follows:

$$C := \{c_1^0, \dots, c_n^0, c_1^1, \dots, c_n^1\} \quad \text{where } n := |V| \quad (1.18)$$

For example, the solution in Fig. 1.7a at page 21a in CMSA format would contain the following set of solution components: $\{c_1^1, c_2^0, c_3^0, c_4^0, c_5^1, c_6^1\}$. Henceforth the standard CMSA algorithm for the MDS based on the generic set of solution components is labeled CMSA_GEN.

1.4.2.1 Solution Construction

For constructing solutions in CMSA_GEN we use exactly the same way as described in the context of CMSA_INT in Sect. 1.4.1.1. In other words, function **ProbabilisticSolutionConstruction**(C) first generates a solution s which consists of a set of vertices. The only difference to the procedure described in Sect. 1.4.1.1 consists in the conversion of s into a solution S of CMSA_GEN. In particular, for each $v_i \in V$

¹⁴ Note that any integer ILP can be transformed into a binary ILP (BIP). This is well known in OR and will be demonstrated employing an example in Chap. 5.

that forms part of s , solution component c_i^1 is added to S , and for each $v_i \in V$ which does not form part of s , solution component c_i^0 is added to S .

1.4.2.2 Sub-instance Solving

A sub-instance C' in CMSA_GEN is solved by adding the following additional constraints to the MDS ILP model (for $i = 1, \dots, |V|$) and solving it with CPLEX:

$$x_i = 0 \quad \text{if } c_i^0 \in C' \text{ and } c_i^1 \notin C' \quad (1.19)$$

$$x_i = 1 \quad \text{if } c_i^0 \notin C' \text{ and } c_i^1 \in C' \quad (1.20)$$

In other words, if C' only contains the solution component c_i^0 corresponding to $x_i = 0$, then the value of x_i is fixed to zero. Similarly, if C' only contains the solution component c_i^1 corresponding to $x_i = 1$, then the value of x_i is fixed to one. Otherwise, if C' contains both solution components c_i^0 and c_i^1 then variable x_i is left free, which means that v_i might, or not, be included in a solution to the sub-instance.

In this context, note that the way of generating the ILP model corresponding to C' in CMSA_GEN is potentially more restrictive than in CMSA_INT, at least in the case of our application to the MDS problem. This can easily be seen as follows. Let us assume that both algorithm variants are in the first iteration (that is, C' is empty), $n_a = 2$ (that is, two solutions are generated per iteration), and the generated solutions are the ones from Fig. 1.7b and c on page 21. Therefore, in CMSA_INT, after solution construction it holds that $C' = \{c_2, c_3, c_4\}$. In the case of CMSA_GEN, it holds that $C' = \{c_1^0, c_2^0, c_2^1, c_3^0, c_3^1, c_4^1, c_5^0, c_6^0\}$. This means that the ILP model corresponding to C' in CMSA_INT is obtained by adding constraints $x_i = 0$ ($i \in \{1, 5, 6\}$) to the MDS ILP model. In contrast, the ILP model corresponding to C' in CMSA_GEN is obtained by also adding constraints $x_i = 0$ ($i \in \{1, 5, 6\}$), but additionally constraint $x_4 = 1$ is added. This means that v_4 must be part of a valid solution to the sub-instance in CMSA_GEN.

1.4.3 Experimental Evaluation

The following algorithms are included in the experimental evaluation presented in this section:

1. GREEDY: A greedy heuristic obtained by executing the heuristic from Algorithm 1.2 on page 22 in a deterministic way.
2. CPLEX: Application of CPLEX 22.1 to each considered problem instance with the default parameter values of CPLEX.
3. CMSA_INT: The standard CMSA algorithm, based on the intuitive way of defining the set of solution components.

Chapter 2

Self-adaptive CMSA



Abstract While the standard CMSA algorithm has proven its utility across a range of different combinatorial optimization problems, certain applications have highlighted its susceptibility to variations in parameter settings. In order to deal with this issue, an innovative self-adaptive variant of the CMSA algorithm, termed ADAPT_CMSA, was developed and will be presented in this chapter. The primary objective is to mitigate the parameter sensitivity that might occur in the standard CMSA variant. The merits of this novel CMSA variant are substantiated through its application to the Minimum Positive Influence Dominating Set (MPIDS) problem and to the Far From Most String (FFMS) problem. Notably, ADAPT_CMSA distinguishes itself from standard CMSA by not presenting the need for a computationally intensive parameter tuning process across subsets within the considered set of problem instances.

2.1 Introduction

The research community on metaheuristics [16] acknowledges a prevalent issue related to an algorithm's high sensitivity to variations in parameter values. In this context, a metaheuristic algorithm is deemed *parameter sensitive* when two conditions are met: (1) the algorithm's performance for particular instances or instance groups significantly relies on specific parameter values, and (2) the necessary parameter values differ notably across distinct instances or instance groups. The research community views excessive sensitivity to parameter settings as an unfavorable aspect.

A noticeable susceptibility to parameter values has also been observed in certain applications of CMSA as documented in the literature. An illustrative case is the initial application of standard CMSA to the Minimum Positive Influence Dominating Set (MPIDS) problem [1], an NP-hard combinatorial optimization problem. In response to this issue, a self-adaptive variant of CMSA named ADAPT_CMSA was proposed in [4]. The primary objective during the development of ADAPT_CMSA was to obtain a CMSA variant that exhibits reduced sensitivity to parameter values. To validate its effectiveness, in this chapter, we present the initial application of

Algorithm 2.1: Pseudo-code of self-adaptive CMSA: ADAPT_CMSA

```

1: input 1: Complete set of solution components  $C$ 
2: input 2: Values for ADAPT_CMSA parameters  $t_{\text{prop}}, t_{\text{ILP}}$ 
3: input 3: Values for solution construction parameters  $\alpha^{\text{LB}}, \alpha^{\text{UB}}, \alpha_{\text{red}}$ 
4:  $S^{\text{bsf}} := \text{GenerateGreedySolution}(C)$ 
5:  $n_a := 1; \alpha_{\text{bsf}} := \alpha^{\text{UB}}; C' := S^{\text{bsf}}$ 
6: while CPU time limit not reached do
7:   for  $i := 1, \dots, n_a$  do
8:      $S := \text{ProbabilisticSolutionConstruction}(C, S^{\text{bsf}}, \alpha_{\text{bsf}})$ 
9:      $C' := C' \cup S$ 
10:  end for
11:   $(S^{\text{ILP}}, t_{\text{solve}}) := \text{SolveSubinstance}(C', t_{\text{ILP}})$  {This function returns two objects: (1) the
    obtained solution ( $S^{\text{ILP}}$ ), (2) the required computation time ( $t_{\text{solve}}$ )}
12:  if  $t_{\text{solve}} < t_{\text{prop}} \cdot t_{\text{ILP}}$  and  $\alpha_{\text{bsf}} > \alpha^{\text{LB}}$  then  $\alpha_{\text{bsf}} := \alpha_{\text{bsf}} - \alpha_{\text{red}}$  end if
13:  if  $f(S^{\text{ILP}}) < f(S^{\text{bsf}})$  then
14:     $S^{\text{bsf}} := S^{\text{ILP}}$ 
15:     $n_a := 1$ 
16:  else
17:    if  $f(S^{\text{ILP}}) > f(S^{\text{bsf}})$  then
18:      if  $n_a = 1$  then  $\alpha_{\text{bsf}} := \min\{\alpha_{\text{bsf}} + \frac{\alpha_{\text{red}}}{10}, \alpha^{\text{UB}}\}$  else  $n_a = 1$  end if
19:    else
20:       $n_a := n_a + 1$ 
21:    end if
22:  end if
23:   $C' := S^{\text{bsf}}$ 
24: end while
25: output:  $S^{\text{bsf}}$ 

```

CMSA to the MPIDS problem. In addition, we describe its application to a second NP-hard combinatorial optimization problem known as the Far From Most String (FFMS) problem.

The results obtained will demonstrate multiple advantages of ADAPT_CMSA over the standard CMSA. Firstly, ADAPT_CMSA eliminates the need for fine-tuning specific parameters for subsets within the designated benchmark set. Following a single round of parameter tuning, ADAPT_CMSA performs effectively across the entire benchmark set, encompassing instances of diverse sizes. Secondly, in the realm of large-scale problem instances, ADAPT_CMSA distinctly outperforms standard CMSA. Even with specialized tuning efforts, standard CMSA cannot match the competitive performance achieved by ADAPT_CMSA in handling such large-scale problem instances.

Note that ADAPT_CMSA has already been used in [2–4, 8].

In function **GenerateGreedySolution**(C) of **ADAPT_CMSA** (see line 4 of Algorithm 2.1 on page 42) for each position j a deterministic choice of the letter with the lowest frequency value is made. Ties are broken randomly.

In contrast, in function **ProbabilisticSolutionConstruction**(C) of our standard **CMSA_INT** (see line 8 of Algorithm 1.1 on page 19), first, a value $r \in [0, 1]$ is drawn uniformly at random for each position $j = 1, \dots, m$. If $r \leq d_{\text{rate}}$ —where $0 \leq d_{\text{rate}} < 1$ is the determinism rate already known from other applications in this book—the letter with the lowest frequency value is chosen in a deterministic way. Otherwise, a letter is chosen randomly (roulette wheel selection) based on probabilities that are proportional to the inverse of the letter frequencies.

Finally, function **ProbabilisticSolutionConstruction**($C, S^{\text{bsf}}, \alpha_{\text{bsf}}$) of **ADAPT_CMSA** (see line 8 of Algorithm 2.1 on page 42) chooses for each position $j = 1, \dots, m$ a letter as follows. First, the following values are defined:

$$q_{j,a} \begin{cases} \frac{\alpha_{\text{bsf}}}{f_{j,a}} & \text{if } c_{j,a} \in S^{\text{bsf}} \\ \frac{(1-\alpha_{\text{bsf}})}{f_{j,a}} & \text{otherwise} \end{cases} \quad (2.16)$$

Subsequently, a letter is chosen for position j by roulette wheel selection based on the following probabilities:

$$p(a) := \frac{q_{j,a}}{\sum_{b \in \Sigma} q_{j,b}} \quad \forall a \in \Sigma \quad (2.17)$$

All three functions, after finishing the construction of a solution in terms of a string (s), transform this solution into a solution S containing for each position j the corresponding solution component. This solution S is then returned to the respective **CMSA** algorithm.

2.4.4 Sub-instance Solving

The way of solving sub-instances C' in the case of the FFMS problem is exactly the same as the one described in Sect. 1.4.2.2 on page 25 in the context of the MDS problem. Therefore, for solving a sub-instance C' by means of the application of **CPLEX** in function **SolveSubinstance**(C', t_{ILP}) of Algorithm 1.1 (page 19) and of Algorithm 2.1 (page 42), the FFMS ILP model is extended by adding the following set of constraints:

$$x_{j,a} = 0 \quad \text{for all } c_{j,a} \in C \setminus C' \quad (2.18)$$

In this way, only solution components (resp. letter-position assignments) that form part of the sub-instance C' may be selected in order to appear in solutions. Finally,

remember that function $\text{SolveSubinstance}(C', t_{\text{ILP}})$ returns the CPLEX solution in terms of a set S^{ILP} of solution components. Moreover, the application of CPLEX is subject to a time limit of t_{ILP} CPU seconds, which means that the solution S^{ILP} returned by the function $\text{SolveSubinstance}(C', t_{\text{ILP}})$ is not necessarily an optimal solution to the sub-instance C' .

2.4.5 Experimental Evaluation

The experimental evaluation for the FFMS problem encompasses the following algorithms:

1. GREEDY: The heuristic obtained by the execution of the algorithm presented in Sect. 2.4.3 (previous section) in a deterministic way.
2. CPLEX: Application of CPLEX 22.1 to each considered problem instance, utilizing the default parameter values of CPLEX.
3. CMSA_INT: The standard CMSA algorithm, making use of the intuitive approach to defining the set of solution components.
4. ADAPT_CMSA: The self-adaptive CMSA approach from this chapter.

As before, CPLEX 22.1 is used—both in standalone mode (CPLEX) and within the CMSA variants—in one-threaded mode. For conducting the experiments we used the IIIA-CSIC in-house high-performance computing cluster of machines equipped with Intel® Xeon® 5670 CPUs having 12 cores of 2.933 GHz and at least 32 GB of RAM.

2.4.5.1 FFMS Benchmark Set

For each combination of $n \in \{100, 200, 300, 400\}$ (number of input strings) and $m \in \{100, 500, 1000\}$ (length of the input strings) exactly 30 problem instances were generated uniformly at random over $\Sigma = \{A, C, T, G\}$. This alphabet of size four was chosen due to the applications of the FFMS in bio-informatics. Moreover, thresholds $t = 0.8m$ and $t = 0.85m$ will be considered for solving all these instances. However, as threshold $t = 0.8m$ proved to be more difficult for our algorithms, the result section below will only show the results obtained for this threshold. In total, the generated benchmark set contains 360 problem instances.

2.4.5.2 Parameter Tuning

Both CMSA_INT and ADAPT_CMSA were tuned with the parameter tuning tool *irace*; see Sect. 1.2.1 on page 12 for a description of *irace*. For this purpose, an additional tuning instance was generated for each combination of n and m , which makes a total of 12 tuning instances, considered both for thresholds $t = 0.8m$ and $t = 0.85m$. The budget of *irace*—that is, the number of algorithm runs allowed